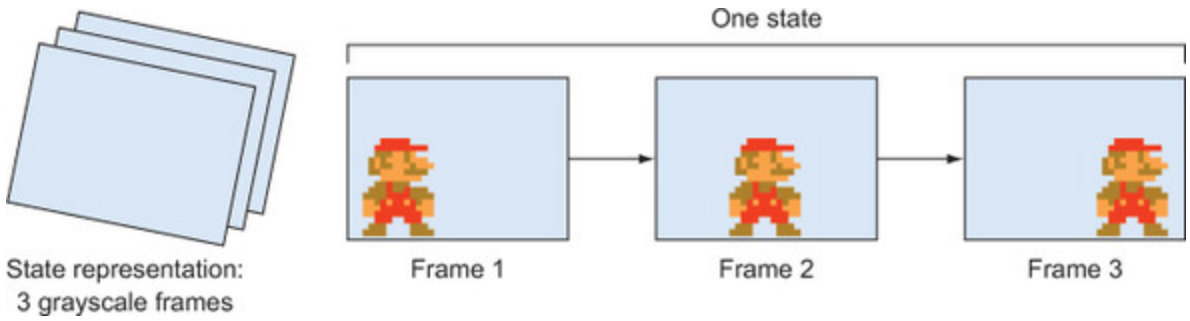
```
>>> plt.imshow(env.render("rgb_array"))
>>> plt.imshow(downscale_obs(env.render("rgb_array")))
```

The downsampled image will look pretty blurry, but it still contains enough visual information to play the game.

We need to build a few other data processing functions to transform these raw states into a useful form. We will not just pass a single 42 × 42 frame to our models; we will instead pass the last three frames of the game (in essence, adding a channel dimension) so the states will be a 3 × 42 × 42 tensor (figure 8.11). Using the last three frames gives our model access to velocity information (i.e., how fast and in which direction objects are moving) rather than just positional information.

Figure 8.11. Each state given to the agent is a concatenation of the three most recent (grayscale) frames in the game. This is necessary so that the model can have access to not just the position of objects, but also their direction of movement.



When the game first starts, we only have access to the first frame, so we prepare the initial state by concatenating the same state three times to get the 3 × 42 × 42 initial state. After this initial state, we can replace the last frame in the state with the most recent frame from the environment, replace the second frame with the old last one, and replace the first frame with the old second. Basically, we have a fixed length first-in-first-out data structure where we append to the right, and the left automatically pops off. Python has a built-in data structure called `deque` in the `collections` library that can implement this behavior when the `maxlen` attribute is set to `3`.

We will use three functions to prepare the raw states in a form that our agent and encoder models will use. The `prepare_state` function resizes